

Crypto Homework 2

Submit the Homework 2 assignment on the course website. The solutions from pairs are accepted.

Submit a single file in **ZIP format (only!)** contains:

- For each exercise a source file. Each source file starts at q_ and his exercise number (for example, q_1).
- Word document led by two task names and ID numbers (should work in pairs only), followed, solutions and examples of input/output for every problem - as required.

Exercise 1

Using AES from a Cryptographic Library (*Python cryptography*)

Encrypt and decrypt the string:

"Homework 2 for the Course Cryptology: Student 1 Full Name, ID and Student 2 Full Name, ID."

using two AES modes: **ECB** and **CBC**.

Display the results in doc file.

Exercise 2

Experimental Cryptanalysis. Avalanche Effect in AES

Perform the following experiment:

1. Take a fixed 128-bit AES key.
2. Encrypt **two plaintexts that differ in exactly one bit**.
3. Compare the two ciphertexts and count how many output bits are different.
4. Repeat this process **5 times** with different random plaintexts.
5. Display all results. How many output bits changed when a single input bit is flipped.

Exercise 3

Brute-Force on a Reduced Key

AES-128 normally uses a **128-bit key**, meaning a full brute-force attack would require checking $2^{128} \approx 3.4 \times 10^{38}$ keys, which is completely infeasible.

To let *practically* implement and measure a brute-force attack, we **artificially reduce the key space**, while still using the real AES-128 algorithm.

The idea:

Instead of using a full 128-bit AES key, we only use **k = 20, 22, or 24 bits**, and fill the remaining bits by repeating the short bitstring.

This produces a key that is **not secure**, but has a small and measurable key space:

- $2^{20} = 1\,048\,576$ keys — can be brute-forced in seconds
- $2^{22} = 4\,194\,304$ keys
- $2^{24} = 16\,777\,216$ keys — noticeably slower

This allows to observe how brute-force time grows.

How to Construct the Reduced Key

Take an integer:

x from 0 to $(2^k - 1)$, where $k = 20 \dots 24$

Example (20 bits):

x = 0b10110011001010101100 # 20 bits

Then convert it into a 128-bit AES key:

K = repeat the 20-bit sequence 7 times and cut to 128 bits

This artificially expands the small key into a full AES-128 key.

Python example:

```
def expand_key(x, bits=20):
    bin_key = format(x, f'0{bits}b')    # convert number to bitstring
    repeated = (bin_key * 7)[:128]      # repeat and trim to 128 bits
    return int(repeated, 2).to_bytes(16, 'big') # convert to 16-byte AES key
```

This construction is artificial but good for brute-force experiments.

Tasks To Do:

1) Choose two values

- **plaintext** (known input)
- **ciphertext**, obtained by encrypting the plaintext with a reduced key

Example:

plaintext = b"Hello student!!!"

ciphertext = AES_encrypt(plaintext, real_key) # real_key is one reduced key

2) Brute-force all possible keys

If the reduced key length is k , the search space is 2^k .

Pseudocode:

for x in range(0, 2^k):

K = expand_key(x) # generate 128-bit AES key

C = AES_encrypt(plaintext, K)

if C == ciphertext:

key_found = x

The program must:

- measure **elapsed time**
- count **number of key checks**

3) Repeat the experiment

Run the brute-force attack for:

k = **20, 22 and 24**

This produces three measurements you can plot.

Plot to Produce

You must create a graph:

X-axis: key size (20, 22 and 24 bits); **Y-axis:** brute-force time (seconds)

GOOD LUCK!!!